

Trying to read 20GB of data, read.csv.sql Produces Errors

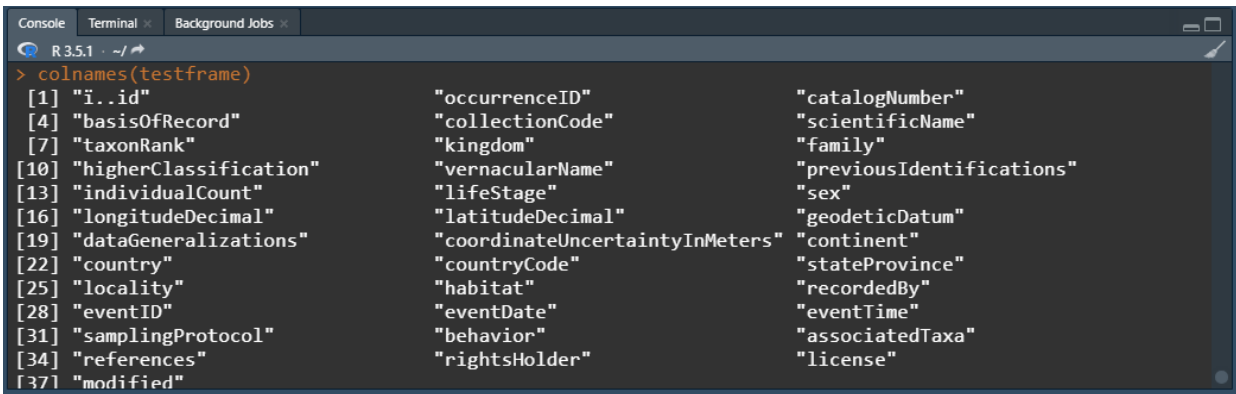
Asked 7 months ago Modified 6 months ago Viewed 224 times  Part of [R Language Collective](#)

I have a 20GB dataset in csv format and I am trying to trim it down with a `read.csv.sql` command.

I am successfully able to load the first 10,000 observations with the following command:

```
testframe = read.csv(file.choose(),nrows = 10000)
```

The column names can be seen in the following picture:



```
> colnames(testframe)
[1] "i..id"                "occurrenceID"          "catalogNumber"
[4] "basisOfRecord"        "collectionCode"        "scientificName"
[7] "taxonRank"            "kingdom"               "family"
[10] "higherClassification" "vernacularName"        "previousIdentifications"
[13] "individualCount"      "lifeStage"             "sex"
[16] "longitudeDecimal"     "latitudeDecimal"        "geodeticDatum"
[19] "dataGeneralizations"  "coordinateUncertaintyInMeters" "continent"
[22] "country"              "countryCode"            "stateProvince"
[25] "locality"             "habitat"                "recordedBy"
[28] "eventID"              "eventDate"              "eventTime"
[31] "samplingProtocol"     "behavior"               "associatedTaxa"
[34] "references"           "rightsHolder"           "license"
[37] "modified"
```

I then tried to build my trimmed down dataset with the following command, and get an error:

```
reduced = read.csv.sql(file.choose(),
  sql = 'select * from file where "country" = "Poland" OR
  country = "Germany" OR country = "France" OR country = "Spain"',
  header = TRUE,
  eol = "\n")
```

The error is: Error in connection_import_file(conn@ptr, name, value, sep, eol, skip) :

RS_sqlite_import: C:\Users\feded\Desktop\AWS\biodiversity-data\occurence.csv line 262 expected 37 columns of data but found 38

Why is it that I can load the first 10,000 observations with ease and problems arise with the second command? I hope you have all the information needed to be able to provide some help on this issue.

[r](#) [dataframe](#) [sqldf](#)

[Share](#) [Improve this question](#) [Follow](#)

edited Oct 6, 2022 at 10:44



[s_baldur](#)

28.8k 4 36 67

asked Oct 6, 2022 at 10:15



[Federico](#)

23 4

1 Is there something weird on line 262? You could check with: `readLines(file.choose(), n = 262)[262]`
– [s_baldur](#) Oct 6, 2022 at 10:49

If you have AWK installed you can pre-process the data using `data.table::fread()` to get the rows for those countries, i.e. `library(data.table); df <- fread("awk -F, '$22 ~ \"country\" || $22 ~ \"Poland\" || $22 ~ \"Germany\" || $22 ~ \"France\" || $22 ~ \"Spain\"' file.csv")` (assuming country is column 22, like in the image you posted) – [jared_mamrot](#) Oct 6, 2022 at 11:18

@jared_mamrot How do I install AWK? I don't know what it is, I searched for packages but didn't find any.
– [Federico](#) Oct 6, 2022 at 11:35

AWK is a programming language: gnu.org/software/gawk/manual/gawk.html. If you're using linux/macOS it comes preinstalled on your system. If you're using windows you can install it following the instructions at: gnu.org/software/gawk/manual/gawk.html#PC-Installation, but if you don't already have it installed, this probably isn't a good solution. Easiest way to find out is to try the command and see if it works – [jared_mamrot](#) Oct 6, 2022 at 11:42 

Have moved my comments to an answer. – [G. Grothendieck](#) Oct 6, 2022 at 18:26

1 Answer

Sorted by: Highest score (default)



▲ Note that with the latest version of all packages read.csv.sql is working again.

3

▼ RSQLite made breaking changes in their interface to SQLite which mean read.csv.sql and any other software that reads files into SQLite from R that used their old interface no longer work. (Other aspects of sqldf still work.)



findstr/grep

If the only reason you are doing this is to cut down the file to the 4 countries indicated perhaps you could just preprocess the csv file like this on Windows assuming that abc.csv is your csv file and that it is in the current directory. Also we have assumed that XYZ is a string in the header.

```
DF <- read.csv(pipe('findstr "XYZ France Germany Poland Spain" abc.csv'))
```

On other platforms use grep:

```
DF <- read.csv(pipe('grep "XYZ|France|Germany|Poland|Spain" abc.csv'))
```

The above could possibly retrieve some extra rows if those words can also appear in fields other than the intended one but if that is a concern then using `subset` or `filter` in R once you have the data in R could be used to narrow it down to just the desired rows.

Other utilities

There are also numerous command line utilities that can be used as an alternative to findstr and grep such as sed, awk/gawk (mentioned in the comments) and utilities specifically geared to csv files such as csvfix (C++), miller (go), csvkit (python), csvtk (go) and xsv (rust).

xsv

Taking xsv as an example, binaries can be downloaded [here](#) and then we can write the following assuming xsv is in current directory or on path. This instructs xsv to extract the rows for which the indicated regular expression matches the country column.

```
cmd <- 'xsv search -s country "France|Germany|Poland|Spain" abc.csv'
DF <- read.csv(pipe(cmd))
```

SQLite command line tool

You can use the SQLite command line program to read the file into an SQLite database which it will create for you. Google for *download sqlite*, download the sqlite command line tools for your platform and unpack it. Then from the command line (not from R) run something like this to create the abc.db SQLite database from abc.csv.

```
sqlite3 --csv abc.db ".import abc.csv abc"
```

Then assuming that the database is in current directory run this in R:

```
library(sqldf)
sqldf("select count(*) from abc", dbname = "abc.db")
```

I am not sure that sqldf is a good choice for such a large file but you can try it

H2

Another possibility if you have sufficient memory to hold the database (possibly after using findstr/grep/xsv or other utility on the command line rather than R) is to then use the H2 database backend to sqldf from R.

If sqldf sees that the RH2 package containing the H2 driver is loaded it will use that instead of SQLite. (It would also be possible to use MySQL or PostgreSQL backends but these are more involved to install so we won't cover them although these are much more likely to be able to handle the large size you have.)

Note that the RH2 driver requires that rJava R package be installed and it requires java itself although java is very easy to install. The H2 database itself is included in the RH2 R driver package so it does not have to be separately installed. Also the first time in a session that you access java code with rJava it will have to load java itself which will take some time but thereafter it will be faster in that session.

```
library(RH2)
library(sqldf)

abc3 <- sqldf("select * from csvread('abc.csv') limit 3") |>
  type.convert(as.is = TRUE)
```

Share Improve this answer Follow

edited Oct 29, 2022 at 13:22

answered Oct 6, 2022 at 18:25



G. Grothendieck

251k 17 198 333

I used `DF <- read.csv(pipe('grep "XYZ|France|Germany|Poland|Spain" abc.csv'))` and it seems to have worked, thank you for all the alternatives you've provided! – Federico Jan 19 at 22:28
